

6강

Service 빈 객체 구현





➤ 기존 Project 복사하기

- sample_pjt-1 를 복사하여 sample_pjt-2로 붙여넣기 한다.
- 단, 프로젝트를 복사하고 나면 settings.gradle, application.properties 세팅을 변경하고 이클립스를 다시 실행한다.

```
settings.gradle ×  
1 rootProject.name = 'sample_pjt-2'  
2 |
```

```
application.properties ×  
1 spring.application.name=sample_pjt-2  
2  
3 # Tomcat 서버를 내장되어 있다.  
4 # 원래 포트번호는 8080인데 => 8090으로 변경함  
5 # Tomcat  
6 server.port=8090  
7  
8 # Dev Tools  
9 spring.devtools.restart.enabled=true  
10  
11 # Thymeleaf #주석  
12 spring.thymeleaf.cache=false  
13 spring.thymeleaf.prefix=classpath:/templates/  
14 spring.thymeleaf.suffix=.html  
15 spring.thymeleaf.check-template-location=true
```



➤ DTO (Data Transfer Object) (데이터 전송 객체) 객체

- 계층(레이어) 간에 데이터를 안전하고 깔끔하게 전달하기 위한 객체
- 데이터가 많다면 DTO 사용 권장!

1) @RequestParam 사용

- 클라이언트에서 전달한 데이터를 **개별 변수**로 받음
- 데이터 개수가 적을 경우 효율적
- 데이터 개수가 많아지면 코드가 길어짐(단점)



```
@PostMapping("/user")
public String createUserService(@RequestParam String name, @RequestParam int age) {
    return "result";
}
```



➤ DTO 객체

2) DTO(Data Transfer Object) 사용

- 여러 개의 데이터를 객체로 묶어 전달
- 스프링이 자동으로 데이터 매핑
- 코드 간결화, 유지보수 용이(장점)

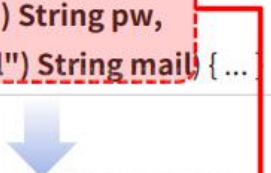
```
public class UserDTO {  
    private String name;  
    private int age;  
    // getter&setter 필요  
}
```

```
@PostMapping("/user")  
public String createUser(UserDto) {  
    return "result";  
}
```



3) MemberController의 signUpConfirm()에서 DTO 사용

```
@PostMapping("/member/signup_confirm")  
public ModelAndView signUpConfirm(  
    @RequestParam("id") String id,  
    @RequestParam("pw") String pw,  
    @RequestParam("mail") String mail) { ... }
```

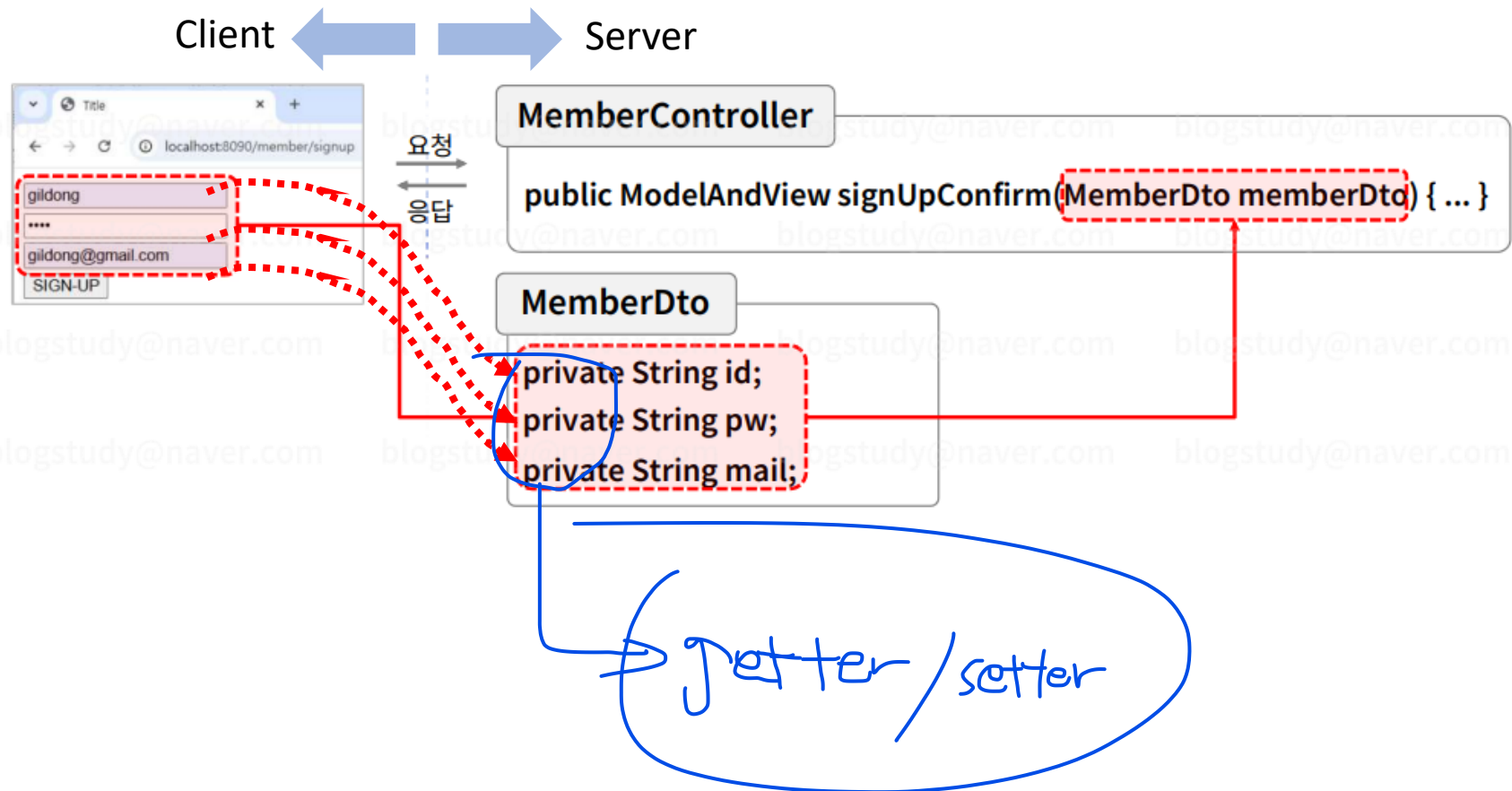


```
@PostMapping("/member/signup_confirm")  
public ModelAndView signUpConfirm(MemberDto memberDto) { ... }
```



➤ DTO 객체

3) MemberController의 signUpConfirm()에서 DTO 사용

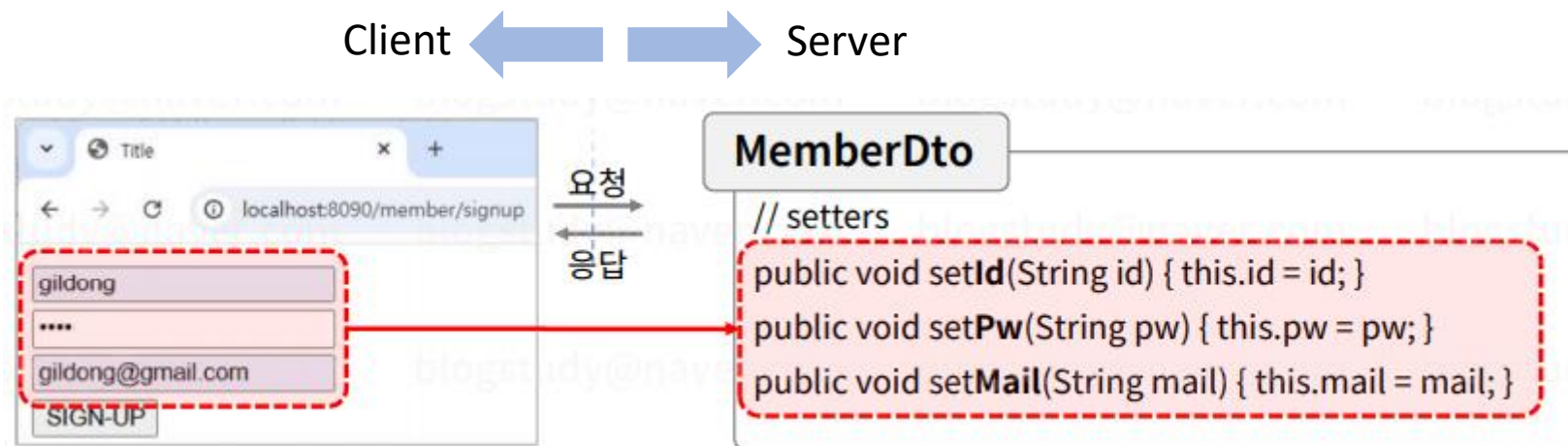




➤ DTO 객체

4) Spring에서 DTO에 데이터 자동 할당 원리

- 스프링은 입력 데이터의 이름과 DTO의 멤버필드 이름을 매칭 후 자동 할당
- 해당 멤버필드의 setter 메서드를 호출하여 값 할당
- 만약 setter가 없는 멤버필드가 있다면 초기화되지 않음



✓ id의 값을 입력하면 필드이름이랑 같은 set id를 알아서 찾아간다.



➤ MemberDTO 간단 예제

- sample_pjt-2 프로젝트에 MemberController 아래 클래스를 생성 후 아래와 같이 작성한다.

```
package com.example.demo.member;
```

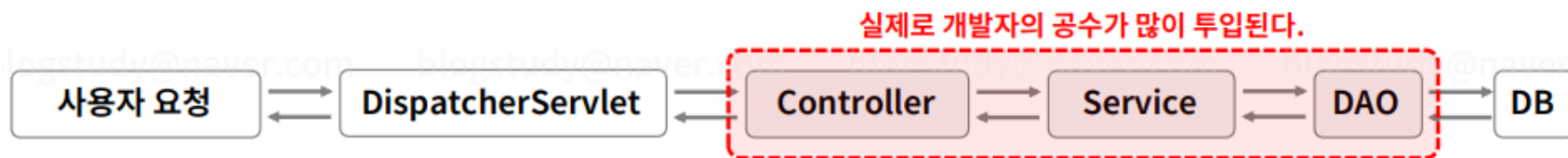
```
public class MemberDTO {  
    private String id;  
    private String pw;  
    private String email;  
  
    // getter : 값을 조회할때 사용한다  
    // setter : 값을 수정할때 사용한다.  
    public String getId() {  
        return id;  
    }  
    public void setId(String id) {  
        this.id = id;  
    }  
    public String getPw() {  
        return pw;  
    }  
    public String getEmail() {  
        return email;  
    }  
    public void setEmail(String email) {  
        this.email = email;  
    }  
    public void setPw(String pw) {  
        this.pw = pw;  
    }  
}
```

```
@PostMapping("/member/sign_confirm")  
public String signipConfirm(MemberDTO mdto, Model model) {  
    System.out.println("id"+mdto.getId());  
    System.out.println("pw"+mdto.getPw());  
    System.out.println("signipConfirm");  
  
    // 현재 시간 구하는 코드  
    SimpleDateFormat sd = new SimpleDateFormat("yyyy-MM-dd HH:mm:ss");  
    Date now = new Date();  
  
    model.addAttribute("now", sd.format(now));  
    model.addAttribute("member_id", mdto.getId());  
    return "signinResult";  
}
```



➤ Service 빈(Bean) 객체 구현

1) Spring에서 Service의 역할



- 비즈니스 로직을 처리하고, 컨트롤러와 데이터 액세스 계층(Repository) 간 중간다리 역할 수행
- [동작 흐름]

Controller: 사용자 요청 처리 → Service 호출

Service: **비즈니스 로직 수행** → 처리 결과 반환

- [역할 예시]

회원가입: ID 중복 체크 → DAO에 신규 회원 등록 요청

상품 구매: 카드 결제 요청 → DAO에 구매 목록 등록 요청



➤ Service 빈(Beans) 객체 구현

2) Service 객체 이용 방법

- 1) 객체 생성 연산자(new)를 이용해서 서비스 객체를 생성하고 이용
- 2) 스프링 설정파일을 이용해서 스프링 컨테이너에 빈(Beans)을, 의존 객체 자동 주입 방법을 이용
- 3) @Configuration과 @Bean을 이용해서 스프링 컨테이너에 빈(Beans)을 생성하고, 의존 객체 자동 주입 방법을 이용
- 4) 서비스 클래스에 @Service 을 명시해서 스프링 컨테이너에 빈(Beans)을 생성하고, 의존 객체 자동 주입 방법을 이용



➤ 객체 생성 연산자(new)를 이용해서 서비스 객체를 생성

- sample_pjt-2 프로젝트에 MemberController 아래 MemberService 클래스를 생성 후 아래와 같이 수정한다. (단, 이방법은 사용하지 않는다)

```
@PostMapping("/member/signup_confirm")
public ModelAndView signUpConfirm(MemberDTO mdto) {
    System.out.println("id : "+mdto.getId());
    System.out.println("pw : "+mdto.getPw());
    System.out.println("email : "+mdto.getMail());
    System.out.println("signUpConfirm");

    MemberService memberservice = new MemberService();
    memberservice.signUpConfirm(mdto);

    // 현재 시간 구하는 코드
    SimpleDateFormat sd = new SimpleDateFormat("yyyy-MM-dd HH:mm:ss");
    Date now = new Date();

    //데이터와 모델을 하나로 합쳐서 클라이언트에 전달하는 ModelAndView
    ModelAndView modelview = new ModelAndView();
    modelview.addObject("now", sd.format(now));
    modelview.addObject("new_member_id", mdto.getId());
    modelview.setViewName("signUpResult");

    return modelview;
}
```

```
1 package com.example.demo.member;
2
3 public class MemberService {
4
5     public void signUpConfirm(MemberDTO mdto) {
6         // TODO Auto-generated method stub
7
8     }
9 }
```



- 객체 생성 연산자(new)를 이용해서 서비스 객체를 생성하고 실행해보기
- 아래처럼 MemberService 클래스에 간단한 log를 입력하고 출력해본다.

localhost:8090/member/signup

id

pw

email

SING-UP

```
1 package com.example.demo.member;
2
3 public class MemberService {
4
5     public void signUpConfirm(MemberDTO mdto) {
6         System.out.println("[MemberService] signUp()");
7     }
8
9     public void signipConfirm(MemberDTO mdto) {
10        System.out.println("[MemberService] signIn()");
11    }
12
13 }
```

2026-01-22T03:46:46.652+09:00
2026-01-22T03:46:46.652+09:00
2026-01-22T03:46:46.924+09:00
2026-01-22T03:46:46.956+09:00
2026-01-22T03:46:46.963+09:00
2026-01-22T03:47:17.538+09:00
2026-01-22T03:47:17.538+09:00
2026-01-22T03:47:17.539+09:00
signinForm()
singUpForm()
id :kjb1045
pw : 11
email : null
signUpComfirm
[MemberService] signUp()



- 객체 생성 연산자(new)를 이용한 Service 객체 생성 및 이용

```
public class MemberService { ... }
```

```
MemberService memberService = new MemberService();
```

- 스프링 설정파일(applicationContext.xml)을 이용한 서비스 객체 생성

```
public class MemberService { ... }
```

```
MemberService memberService = new MemberService();
```

```
<beans xmlns="http://www.springframework.org/schema/beans"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://www.springframework.org/schema/beans
    http://www.springframework.org/schema/beans/spring-beans.xsd">

  <bean id="memberService" class="com.office.ch04_pjt_01.member.MemberService" />
</beans>
```



➤ 객체 생성 연산자(new)를 이용해서 서비스 객체를 생성

- sample_pjt-2 프로젝트에 src -> main -> resources -> applicationContext.xml 파일 생성후 아래 코드를 작성하여 Bean객체를 생성한다. 단, 이 방식은 Spring 레거시한 프로젝트에서 많이 사용하던 방식으로 요즘 Spring Boot에서는 사용하지 않는다.

```
*applicationContext.xml x
1 <beans xmlns="http://www.springframework.org/schema/beans"
2     xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
3     xsi:schemaLocation="http://www.springframework.org/schema/beans
4         http://www.springframework.org/schema/beans/spring-beans.xsd">
5
6     <bean id="memberService" class="com.example.demo.member.MemberService" />
7
8 </beans>
```



➤ @ImportResource를 이용한 의존 객체 자동 주입

```
@SpringBootApplication
```

```
@ImportResource("classpath:applicationContext.xml") // 스프링 IoC 컨테이너에 등록
```

```
public class Ch04Pjt01Application { ... }
```

```
@Autowired
```

```
MemberService memberService;
```



➤ classpath란?

- 애플리케이션이 실행되는 동안 필요한 리소스와 클래스 파일을 찾기 위해 사용되는 경로를 의미합니다.
- 리소스 로딩: application.properties 같은 설정 파일, 정적 리소스(static, templates 디렉토리 등), XML 파일 등을 로딩합니다.
- 클래스 로딩: 애플리케이션에서 필요한 클래스 파일(.class)을 JVM이 로드할 때 검색하고 로딩합니다.

➤ @Configuration을 이용한 서비스 객체 생성

```
public class MemberService { ... }
```

```
<bean id="memberService" class="com.office.ch04_pjt_01.member.MemberService" />
```

```
@Configuration
public class AppConfig {
    @Bean
    public MemberService memberService() {

        return new MemberService();
    }
}
```



- @Configuration과 @Bean을 이용해서 스프링 컨테이너에 빈(Bean)객체 생성하고 자동 주입
 - sample_pjt-2 프로젝트에 MemberController 아래 AppConfig 클래스를 생성 후 아래와 같이 작성한다.

```
AppConfig.java x
1 package com.example.demo.member;
2
3 import org.springframework.context.annotation.Bean;
4 import org.springframework.context.annotation.Configuration;
5
6 @Configuration
7 public class AppConfig {
8
9     //멤버 서비스를 Bean객체로 생성하는 작업
10    @Bean
11    public MemberService memberService() {
12        return new MemberService(); //Ioc 컨테이너에 Bean객체로 탑재시킨다.
13        //Ioc컨테이너에 MemberService클래스가 탑재된다.
14    }
15 }
```




- @Configuration과 @Bean을 이용해서 스프링 컨테이너에 빈(Beans) 객체 생성하고 자동 주입
- sample_pjt-2 프로젝트에 MemberController 아래와 같이 수정한다.

```
package com.example.demo.member;

import java.text.SimpleDateFormat;

@Controller // Controller이라는 Bean객체가 생성되도록 어노테이션처리 한다.
public class MemberController {

    // MemberController가 직접 MemberService를 생성하지 않고
    // 스프링 컨테이너가 만든 MemberService 객체를 주입받는다
    @Autowired
    MemberService memberService;
```

```
@PostMapping("/member/signup_confirm")
public ModelAndView signUpConfirm(MemberDTO mdto) {
    System.out.println("id : "+mdto.getId());
    System.out.println("pw : "+mdto.getPw());
    System.out.println("email : "+mdto.getMail());
    System.out.println("signUpConfirm");

    // MemberService memberservice = new MemberService();
    memberService.signUpConfirm(mdto);
```



- @Configuration과 @Bean을 이용해서 스프링 컨테이너에 빈(Bean)객체 생성하고 자동 주입 실행결과

localhost:8090/member/signup

지마켓 쿠팡! G마켓 - 쇼핑을 바... Gmail

kjb1045

....

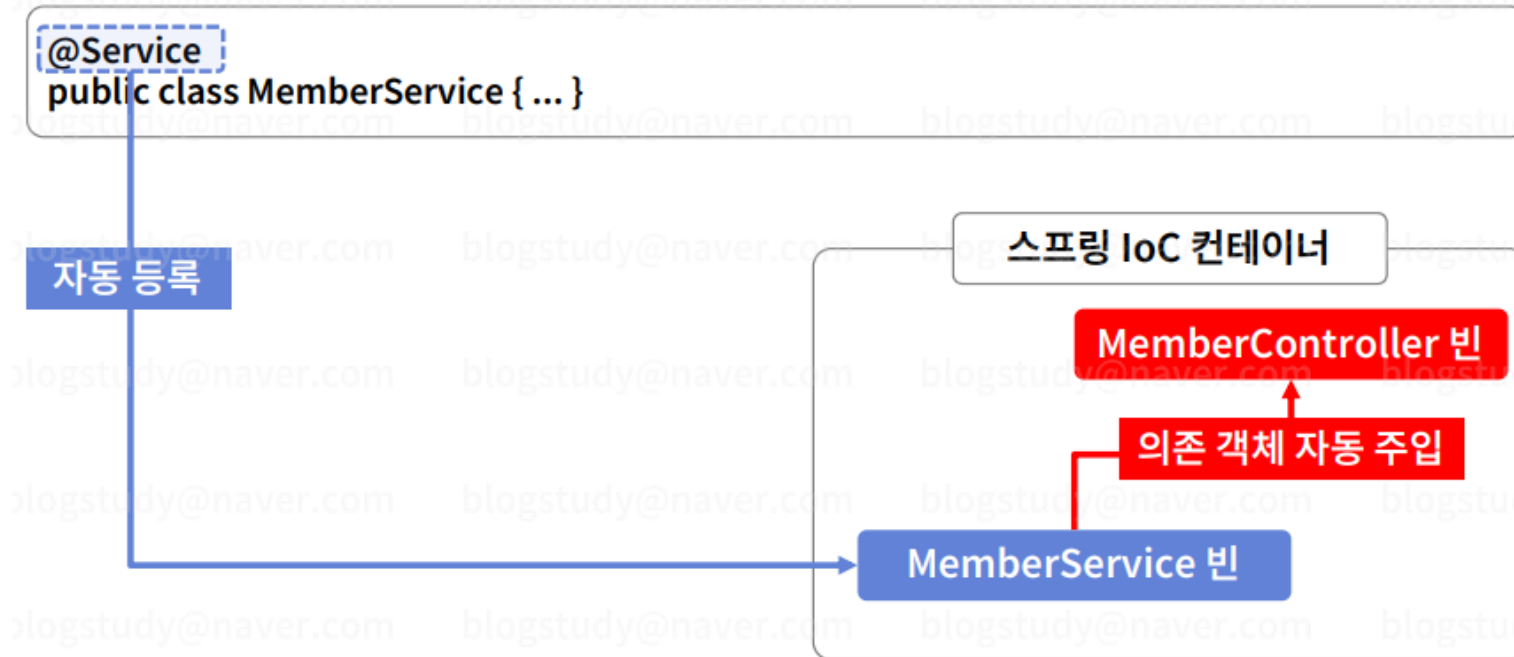
kjb1045@naver.com

SING-UP

```
Problems @ Javadoc Declaration Console X Grad
sample_pjt-2 - SamplePjt1Application [Spring Boot App] C:\WProgram
2026-01-22T04:08:30.550+0
2026-01-22T04:08:30.587+0
2026-01-22T04:08:30.595+0
2026-01-22T04:08:50.308+0
2026-01-22T04:08:50.309+0
2026-01-22T04:08:50.311+0
singUpForm()
id :kjb1045
pw : 1111
email : null
signUpComfirm
[MemberService] signUp()
```



➤ @Service를 이용한 서비스 객체 생성





➤ @Service를 명시해서 스프링 컨테이너에 빈(Bean)을 생성하고, 의존 객체 자동 주입

- AppConfig 부분을 아래와 같이 수정한다.

```
1 package com.example.demo.member;
2
3 import org.springframework.context.annotation.Bean;
4
5
6 // @Configuration
7 public class AppConfig {
8
9     // 멤버 서비스를 Bean객체로 생성하는 작업
10    // @Bean
11    public MemberService memberService() {
12        return new MemberService(); // IoC 컨테이너에 Bean객체로 탑재시킨다.
13        // IoC컨테이너에 MemberService클래스가 탑재된다.
14    }
15 }
```

- MemberService 클래스에 @Service 어노테이션을 입력하고 아래와 같이 수정한다.

```
1 package com.example.demo.member;
2
3 import org.springframework.stereotype.Service;
4
5 @Service
6 public class MemberService {
7
8     public void signUpConfirm(MemberDTO mdto) {
9         System.out.println("[MemberService] signUp()");
10    }
11
12
13    public void signipConfirm(MemberDTO mdto) {
14        System.out.println("[MemberService] signIn()");
15    }
16
17
18 }
```

6강

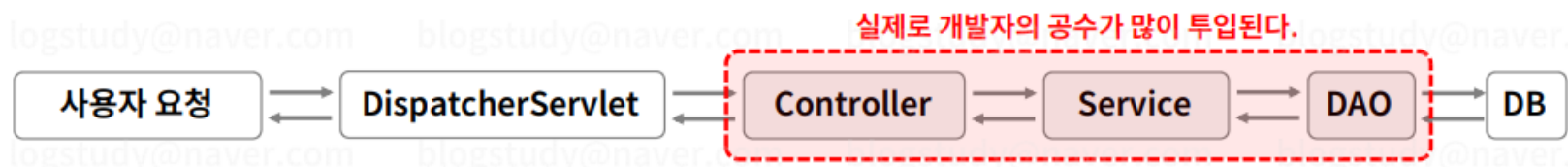
DAO 빈 객체 구현





➤ DAO(Data Access Object) 빈(Bean) 객체 구현

- 데이터베이스(DB)에 접근하는 역할만 담당하는 객체이다.
- DAO도 Service처럼 스프링 IoC 컨테이너에 빈 객체로 생성 후 자동 주입 가능하다.



1) DAO 빈 생성 방법

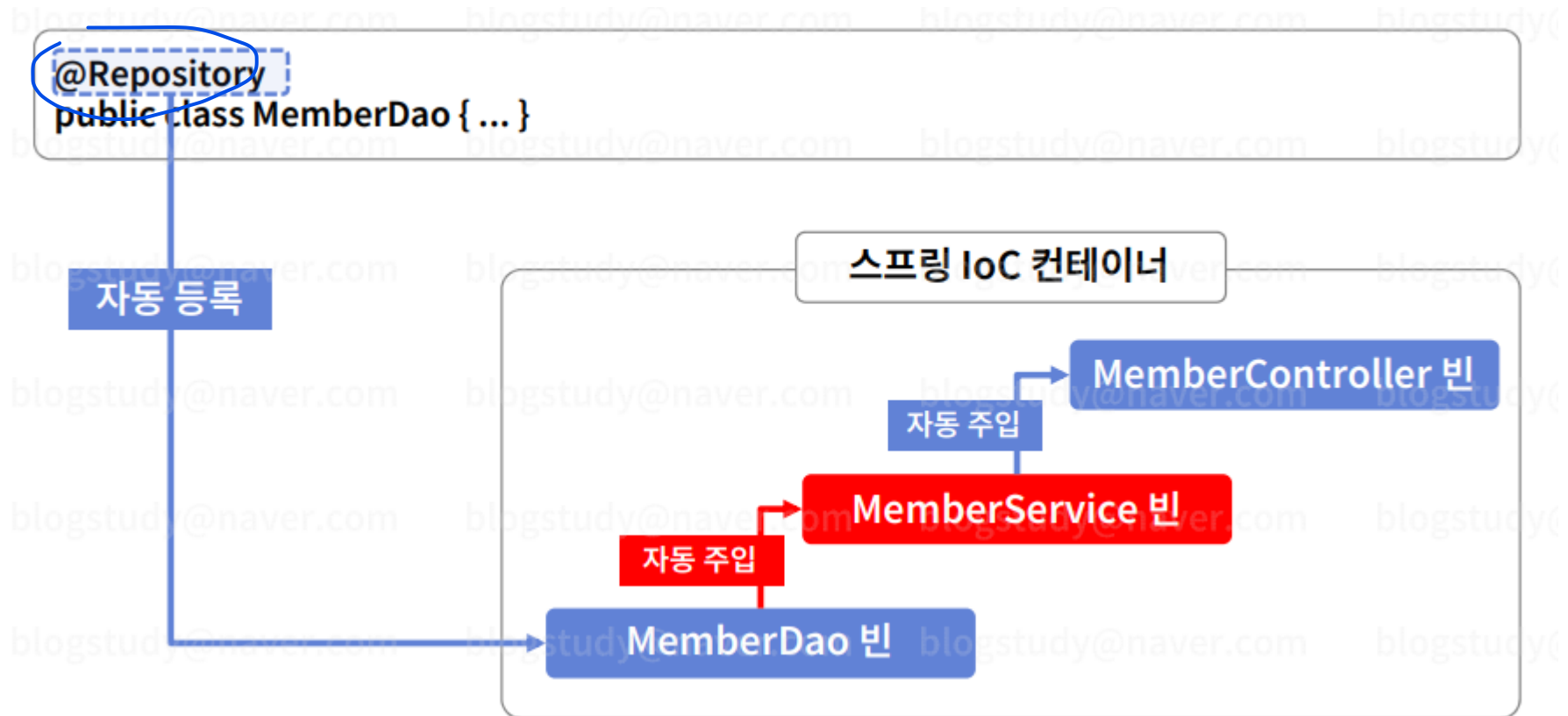
- @Repository 사용 저장소
- 기타 방법도 존재 (예: @Component, @Service 등)

2) 의존 객체 자동 주입

- @Autowired를 사용하여 필요한 곳(예: MemberService)에 DAO 주입



➤ DAO 빈(Beans) 객체 구현





➤ MemberDAO 간단 예제

- sample_pjt-2 프로젝트에 MemberController 아래 MemberDAO 클래스를 생성 후 아래와 같이 작성한다.

```
import java.util.Map;

import org.springframework.stereotype.Repository;
import org.springframework.stereotype.Service;

//@Service
@Repository
public class MemberDAO {
    //우선 DB가 존재하지 않으니까 DB역활을 하는 Map<>생성
    private Map<String, MemberDTO> memberDB = new HashMap<>();

    public void insertMember(MemberDTO mdto) {
        System.out.println("[MemberService] insertMember()");
    }

    public void selectMember(MemberDTO mdto) {
        System.out.println("[MemberService] selectMember()");
    }
}
```




➤ MemberDAO 간단 예제

- sample_pjt-2 프로젝트에 MemberController 아래 MemberService 클래스를 아래와 같이 수정하고 실행한다.

```
package com.example.demo.member;

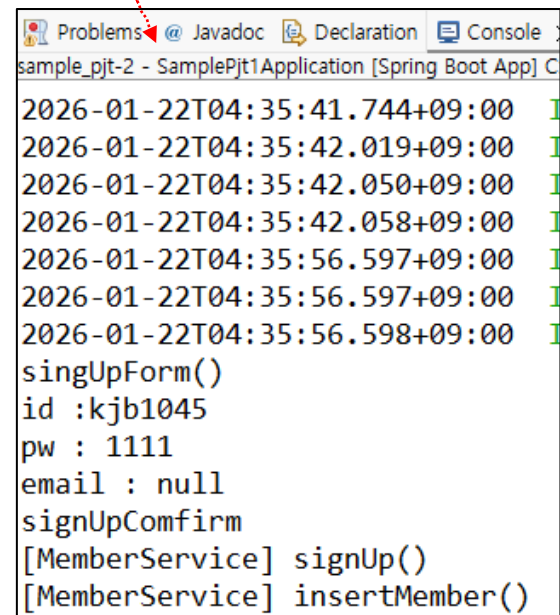
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

@Service
public class MemberService {

    //MemberDAO클래스를 MemberService클래스에 의존성 객체로 주입함
    @Autowired
    MemberDAO memberDao;

    public void signUpConfirm(MemberDTO mdto) {
        System.out.println("[MemberService] signUp()");
        memberDao.insertMember(mdto);
    }

    public void signipConfirm(MemberDTO mdto) {
        System.out.println("[MemberService] signIn()");
        memberDao.selectMember(mdto);
    }
}
```



```
Problems Javadoc Declaration Console
sample_pjt-2 - SamplePjt1Application [Spring Boot App] C
2026-01-22T04:35:41.744+09:00 I
2026-01-22T04:35:42.019+09:00 I
2026-01-22T04:35:42.050+09:00 I
2026-01-22T04:35:42.058+09:00 I
2026-01-22T04:35:56.597+09:00 I
2026-01-22T04:35:56.597+09:00 I
2026-01-22T04:35:56.598+09:00 I
singUpForm()
id :kjb1045
pw : 1111
email : null
signUpComfirm
[MemberService] signUp()
[MemberService] insertMember()
```



➤ MemberDAO 간단 예제

- sample_pjt-2 프로젝트에 MemberController 아래 MemberDAO 클래스를 아래와 같이 수정하고 실행한다.

```
@Repository
public class MemberDAO {
    //우선 DB가 존재하지 않으니까 DB역할을 하는 Map<>생성
    private Map<String, MemberDTO> memberDB = new HashMap<>();

    public void insertMember(MemberDTO mdto) {
        System.out.println("[MemberService] insertMember()");

        System.out.println("id: "+mdto.getId());
        System.out.println("pw: "+mdto.getPw());
        System.out.println("email: "+mdto.getEmail());

        //memberDB에 put메소드 이용해서 삽입한다.
        memberDB.put(mdto.getId(), mdto);
        printMember();
    }

    public void selectMember(MemberDTO mdto) {
        System.out.println("[MemberService] selectMember()");
    }
}
```

```
// Map<> 데이터 출력하는 메소드
public void printMember() {
    System.out.println("[MemberDao]");

    for(String key : memberDB.keySet()) {
        MemberDTO mdto = memberDB.get(key);

        System.out.println("id: "+mdto.getId());
        System.out.println("pw: "+mdto.getPw());
        System.out.println("email: "+mdto.getEmail());
    }
    System.out.println("-----");
}
```

```
[MemberDao]
id: bbb
pw: 2222
email: fire4000@naver.com
-----
id: kjb1045
pw: 1111
email: kjb1045@naver.com
-----
```

6강

로그인 Service, DAO 빈 객체 구현





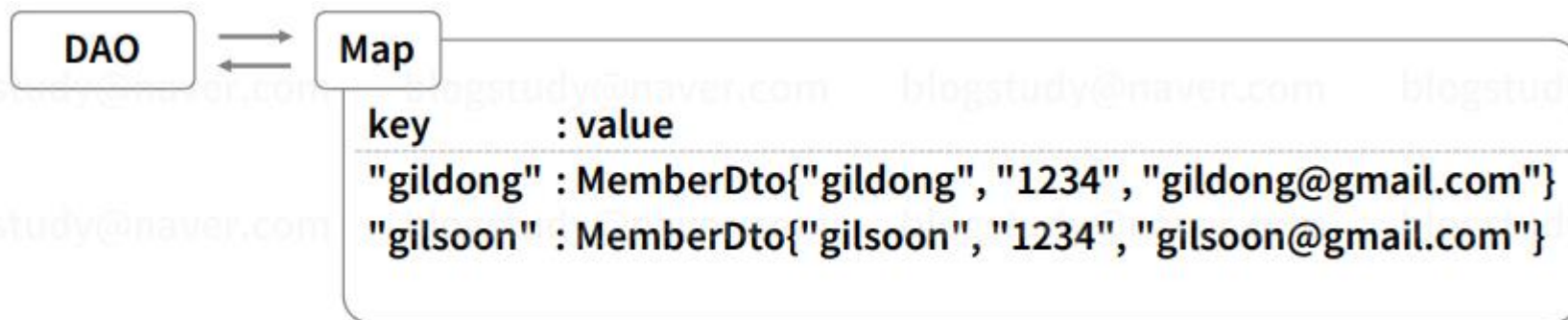
➤ 로그인 Service, DAO 빈(Bean) 객체 구현

실제로 개발자의 공수가 많이 투입된다.



DAO 구현 방법

- DAO 의 경우 데이터베이스와 통신하는 객체로 실제로는 데이터베이스와 연결
- 아직 데이터베이스에 대한 학습을 하지 않은 관계로 우선은 Map 타입의 자료형을 이용해서 DB를 대신 함





➤ MemberDAO 간단 예제

- sample_pjt-2 프로젝트에 MemberController 아래 MemberDAO 클래스를 아래와 같이 수정하고, signUp 폼에서 두 명의 회원을 가입한 후 signin 폼에서 로그인을 실행해본다.

```
public void selectMember(MemberDTO mdto) {  
    System.out.println("[MemberService] selectMember()");  
  
    //회원의 id,pw가 일치하는지 처리  
    MemberDTO loginMember = memberDB.get(mdto.getId());  
  
    if(loginMember != null && mdto.getPw().equals(loginMember.getPw())) {  
        System.out.println("login Success!!");  
        System.out.println("id"+loginMember.getId());  
        System.out.println("pw"+loginMember.getPw());  
        System.out.println("email"+loginMember.getEmail());  
    }else {  
        System.out.println("login Fail!!");  
    }  
}
```

Problems @ Javadoc Declaration Console ×
sample_pjt-2 - SamplePjt1Application [Spring Boot App] C:\W

[MemberDao]
id: bbb
pw: 3333
email: fire4000@naver.com

id: kjb1045
pw: 1111
email: kjb1045@naver.com

[MemberService] signIn()
[MemberService] selectMember()
login Success!!
idkjb1045
pw1111
emailkjb1045@naver.com
signinForm()
idbbb
pw1111
signipComfirm
[MemberService] signIn()
[MemberService] selectMember()
login Fail!!



➤ MemberDAO 간단 예제

- sample_pjt-2 프로젝트에 MemberController 아래 MemberDAO 클래스에 작성한 로그인 체크기능은 DAO클래스 보다는 Service인 비즈니스로직이기에 MemberService 클래스로 그 기능을 이동시켜 리팩토링 한다.

```
*MemberDAO.java X
27 public MemberDTO selectMember(MemberDTO mdto) {
28     System.out.println("[MemberService] selectMember()");
29
30     //회원의 id, pw가 일치하는지 처리
31     MemberDTO loginMember = memberDB.get(mdto.getId());
32
33     return loginMember;
34 }
35
```

```
*MemberService.java X
6 @Service
7 public class MemberService {
8
9     //MemberDAO클래스를 MemberService클래스에 의존성 객체로 주입함
10    @Autowired
11    MemberDAO memberDao;
12    public void signUpConfirm(MemberDTO mdto) {
13        System.out.println("[MemberService] signUp()");
14        memberDao.insertMember(mdto);
15    }
16
17    public void signipConfirm(MemberDTO mdto) {
18        System.out.println("[MemberService] signIn()");
19        // memberDao.selectMember(mdto);
20
21        MemberDTO loginMember = memberDao.selectMember(mdto);
22
23        if(loginMember != null && mdto.getPw().equals(loginMember.getPw())) {
24            System.out.println("login Success!!");
25            System.out.println("id"+loginMember.getId());
26            System.out.println("pw"+loginMember.getPw());
27            System.out.println("email"+loginMember.getEmail());
28        }else {
29            System.out.println("login Fail!!");
30        }
31
32    }
33 }
```

연습문제





➤ 여기서 잠깐 !

- sample_pjt03 -> book 패키지 안에 필요한 클래스를 작성하고 아래 <출력 결과>와 동일하게 화면에 출력 되도록 도서 대여 프로그램을 작성하시오.

조건

- ① 도서 정보를 입력받는다.
 - 도서명
 - 저자
 - ISBN
- ② 대여자 이름을 입력받는다.
- ③ 도서 대여는 1권 이상 가능하도록 처리한다.
- ④ 대여된 도서 정보는
 - 도서명
 - 저자
 - ISBN
 - 대여자 이름을 모두 출력한다.
- ⑤ DTO, DAO, Service, Controller 구조를 사용하여 작성한다.
- ⑥ 출력 화면은 HTML 페이지로 구성한다.



<출력 결과>

도서 대여 정보

도서명 : 스프링 부트 입문

저자 : 김영한

ISBN : 9791158392236

대여자 : 이순신

도서 대여 정보

도서명 : 자바의 정석

저자 : 남궁성

ISBN : 9788994492032

대여자 : 홍길동

<프로젝트 구조>

com.green.book

├ dto

└ BookDTO.java

├ dao

└ BookDAO.java

├ service

└ BookService.java

└ controller

└ BookController.java

resources/templates

├ bookAdd.html

├ bookList.html

└ bookRent.html